

AegisFlow EHS

Factory Compliance & Alert Escalation System

2-Week Development Schedule — Team of 2

1. Dataset & Reference Material

Video Dataset (Kaggle): <https://www.kaggle.com/datasets/trnhhnggiang/videodataset-for-safe-and-unsafe-behaviours>

Compliance Policy Document: KMP-OHS-POL-001 — Occupational Health & Safety Compliance Policy Manual (provided PDF, defines the 4 behavioral domains, severity signals, and reporting requirements).

Assignment Reference: Factory Compliance & Alert Escalation System — Intern Take-Home Assignment (defines the 5 required functional modules and deliverables).

2. Team Structure

This plan is designed for a 2-person team working in parallel with a daily sync. Person A focuses on the AI/CV side (detection + policy parsing/severity). Person B focuses on the backend/systems side (escalation, reports, dashboard). Both integrate together on shared milestone days (Day 7, Day 12).

3. Feature Breakdown by Module

Module 1 — Detection Engine (*Owner: Person A (Computer Vision / AI)*)

- YOLOv8-Nano for person/object/forklift detection on video frames.
- HSV color-space analysis to identify green vs red-black safety vest.
- Contour matching for open vs closed electrical panel cover state.
- Block counting logic for forklift load (2 = safe, 3+ = overload).
- Google Gemini Vision fallback for ambiguous/borderline cases.
- Output: structured detection record — clip_id, timestamp, rule breached, description, zone.

Module 2 — Policy-Grounded Severity Matrix (*Owner: Person A (NLP / Policy Parsing)*)

- Parse KMP-OHS-POL-001 PDF and extract the 4 behavior domains + observable indicators.
- Map policy callouts (WARNING vs CRITICAL SAFETY NOTICE) and frequency language to LOW / MEDIUM / HIGH / CRITICAL tiers.
- Severity assignment is derived from policy text, not hardcoded.
- Traceability: every rule linked back to its policy section (e.g., Section 3.3.2).

Module 3 — Escalation Pipeline (*Owner: Person B (Backend)*)

- LOW / MEDIUM violations → persistent database log only.
- HIGH / CRITICAL violations → real-time alert (WebSocket/pub-sub) + database log.
- Handles multiple simultaneous violations of different severities within one clip.

Module 4 — Automated Report Generation (*Owner: Person B (Backend)*)

- Auto-generated, immutable record per violation with required fields: event_id, timestamp, clip_id, zone, behavior_class, policy_rule_ref, event_description, severity, escalation_action.
- Export formats: JSON and/or CSV, append-only audit log.

Module 5 — Operations Dashboard (*Owner: Person B (Frontend / Backend)*)

- View A — Live Feed Monitor: processed clip playback with color-coded status overlay.
- View B — Alert Timeline Stream: real-time chronological event feed, visual alert for High/Critical.
- View C — Historical Log & Export: filter by date range, severity, behavior class; export button (CSV/JSON).

4. Day-by-Day Schedule (14 Days)

Both team members hold a short daily sync (15–20 minutes) to align interfaces between modules. Weekly milestones fall on Day 7 (mid-point integration) and Day 14 (final submission).

Day	Focus	Person A — AI / Detection / Policy	Person B — Backend / Escalation / Dashboard	Deliverable / Milestone
Day 1	Setup & Planning	Repo setup, environment setup, download dataset & policy PDF, read assignment thoroughly, finalize architecture & module boundaries together.	Set up FastAPI backend skeleton, SQLite schema draft (event_id, timestamp, clip_id, zone, behavior_class, policy_rule_ref, severity, escalation_action).	Repo structure ready, dataset downloaded, architecture diagram finalized.
Day 2	Policy Parsing (RAG)	Build policy-parsing pipeline: extract text from KMP-OHS-POL-001 PDF, chunk sections, extract the 4 behavior classes + observable indicators using NLP/RAG approach.	Design database models (SQLAlchemy) and CRUD layer for violation records; scaffold FastAPI routes.	Structured JSON of policy rules (behavior_class → indicator → severity signal).
Day 3	Severity Matrix	Map policy language (WARNING vs CRITICAL SAFETY NOTICE callouts, frequency data) into LOW/MEDIUM/HIGH/CRITICAL severity tiers per behavior class, grounded in policy text.	Implement escalation pipeline skeleton: routing logic (LOW/MED → DB log, HIGH/CRIT → alert trigger) as a callable module.	Severity matrix module + escalation routing logic (unit-testable).
Day 4	Detection Engine — Setup	Set up YOLOv8-Nano for object/person detection; test on sample Kaggle video clips; set up OpenCV frame extraction pipeline.	Build WebSocket / pub-sub layer for real-time alert push; test with dummy events.	YOLOv8 running on sample clips, WebSocket alert channel working end-to-end (dummy data).
Day 5	Detection — Walkway + Vest	Implement Safe Walkway Violation detection (person position vs green boundary) and Unauthorized Intervention detection (HSV color analysis for vest color).	Implement report generator module (Module 4): auto-writes structured JSON/CSV record per violation with all required fields.	2 of 4 behavior classes detectable; report generator producing valid records.
Day 6	Detection — Panel + Forklift	Implement Opened Panel Cover detection (contour/state matching) and Forklift Overload detection (block counting, threshold = 2 safe / 3+ unsafe).	Connect report generator to database; implement historical log query layer with filters (date, severity, behavior class).	All 4 behavior classes detectable; historical log filtering functional.

Day	Focus	Person A — AI / Detection / Policy	Person B — Backend / Escalation / Dashboard	Deliverable / Milestone
Day 7	Mid-Point Integration	Both: Integrate detection engine output → severity matrix → escalation pipeline → report generator as one working pipeline. Run on full clip set, fix mismatches.	Both: Integrate detection engine output → severity matrix → escalation pipeline → report generator as one working pipeline. Run on full clip set, fix mismatches.	End-to-end pipeline (video in → compliance report out) working for at least 1 clip per behavior class.
Day 8	Gemini Vision Fallback	Add Gemini Vision VLM fallback for ambiguous/borderline detections (e.g., forklift 2 vs 3 blocks); log confidence + fallback usage.	Start Dashboard — View A: Live Feed Monitor (processed clip playback + status indicator overlay, color-coded by severity).	Fallback logic reduces borderline misclassification; Live Feed view renders clip + status.
Day 9	Dashboard — Alerts	Run detection pipeline across full Kaggle dataset, log accuracy/failure notes for README ("known limitations").	Dashboard — View B: Alert Timeline Stream (real-time chronological event feed via WebSocket) + visual/audible strobe for High/Critical.	Alert timeline live-updating on dashboard; critical alerts visibly flagged.
Day 10	Dashboard — History & Export	Support Person B with API contracts for historical log endpoint; begin writing policy-parsing + model-selection section of README.	Dashboard — View C: Historical Log & Export (filter by date/severity/behavior class, export CSV/JSON button).	All 3 dashboard views functional and connected to backend.
Day 11	Multi-Violation Handling	Handle edge case: multiple simultaneous violations of different severities in one clip — ensure each is logged/escalated independently.	Polish escalation pipeline for concurrent events; add basic auth/config for facility routing preferences (optional).	Pipeline correctly handles multi-violation clips without dropping events.
Day 12	Full System Testing	Run full regression across all provided clips; note detection accuracy, false positives/negatives per class.	Run full regression on backend + dashboard: DB integrity, export correctness, WebSocket stability under load.	Bug list closed; system stable across full dataset.
Day 13	Documentation & Cleanup	Write README sections: architecture overview, policy-parsing approach, severity-mapping rationale, model choices, known limitations.	Write README sections: setup/run instructions, dashboard usage, API reference; clean repo structure to match required layout.	Complete, navigable README.md; repo matches recommended structure.
Day 14	Demo & Submission	Record demo video walkthrough (shared task); final review of policy-grounding traceability (rule → policy section).	Final QA pass, push to GitHub/GitLab, verify repo is accessible to reviewers, double-check submission checklist.	Repository submitted, demo video linked, checklist fully verified.

5. Week Summary

- Week 1 (Day 1–7): Setup, dataset ingestion, policy parsing, severity matrix design, detection engine built out for all 4 behavior classes, escalation + report skeleton, first end-to-end integration.
- Week 2 (Day 8–14): Gemini Vision fallback, full dashboard (3 views), multi-violation handling, full regression testing, documentation, demo video, final submission.

6. Final Deliverables Checklist

- Git repository with README.md (setup, architecture, policy-parsing approach, severity-mapping rationale).
- Working detection engine covering all 4 behavior classes, grounded in policy document.
- Severity matrix + escalation pipeline (LOW/MED → log, HIGH/CRIT → real-time alert + log).
- Automated report generation (JSON/CSV) with all required fields.
- Functional dashboard: Live Feed Monitor, Alert Timeline Stream, Historical Log & Export.
- Demo video walkthrough (recommended).